

SLIDING WINDOW

Day 2 — Two Must-Solve Problems

The Universal 4-Step Framework, Fixed Window vs Variable Window, and the Hire & Fire Intuition — with full C++ code

DSA Pattern Series — Sliding Window, Part 2

Covers: Maximum Sum Subarray of Size K + Minimum Size Subarray Sum

Compiled from video transcript + added explanations, tables & code

Table of Contents

1. Quick Recap — Is It Sliding Window?

Before solving anything, run every new problem through this 3-question filter (covered in detail in Day 1):

1. Is it an Array, List, or String question? (Never Linked List.)
2. Does it involve a Subarray / Substring — i.e. CONTIGUOUS elements (not a subsequence)?
3. Is the question asking for Max, Min, Longest, Shortest, Sum, Count, Average, At Most K, At Least K, or Exactly K?

✓ Golden Rule (unchanged from Day 1)

If all three are YES, it is almost certainly a Sliding Window problem.

It might technically be solvable some other way, but Sliding Window will be the most optimal approach.

Today's twist: once you know it IS Sliding Window, that's only 50% of the job. The real skill is figuring out HOW to apply it — and that is exactly what today's 4-step framework gives you.

2. The Universal 4-Step Framework

Every single Sliding Window problem — no matter how easy or hard — is solved using exactly these four steps. Memorize this; nothing outside of it will ever be asked.

Step 1 — Identify the Pattern

Run the 3-question filter from Section 1. Confirm it really is a Sliding Window problem before writing a single line of code.

Step 2 — Determine the Window Type

- **Fixed Window:** the problem gives you an exact size/length (e.g. "subarray of size K"). The window NEVER changes size — it only slides.
- **Variable Window:** no size is given. The window grows and shrinks on its own based on a condition (e.g. "smallest subarray whose sum $\geq$ target").

Step 3 — Find the Information of the Starting Window

"Information" simply means the meaningful answer that window currently represents — its sum, its max, its min, its count of something, etc. Whatever your very first window is (however small), calculate this information for it directly. There's no shortcut for the very first window — you must compute it the slow way once.

Step 4 — Slide the Window & Update the Information

Move the window forward (slide it, expand it, or shrink it — whatever the problem needs) and update the information using the PREVIOUS window's result, instead of recalculating from scratch. Repeat this until you reach the end of the array, run out of window, or already have your answer.

The 4 Steps in One Line

1. Identify → 2. Fixed or Variable? → 3. Info of the first window → 4. Slide + update info, repeat.

3. Problem 1 — Maximum Sum Subarray of Size K (Fixed Window)

🔗 Problem Statement

Given an array and a number K, find the maximum sum among all contiguous subarrays of size K.

Array: [100, 200, 300, 400] K = 2

3.1 Applying Step 1 — Identify

Check	Is it true here?
Array / String question?	✓ Yes — it's an array
Subarray / Substring (contiguous)?	✓ Yes — "subarray of size K"
Asking for Max / Min / Sum / etc.?	✓ Yes — "maximum sum"

All three boxes ticked → confirmed Sliding Window problem.

3.2 Applying Step 2 — Window Type

The problem hands us an explicit size: K = 2. Since the size is given and never needs to change, this is a FIXED window.

3.3 Why Reuse Instead of Recompute? (The Bank Balance Analogy)

Imagine 5 people hand you money one by one: ₹5, ₹4, ₹2, ₹10, ₹5. You count it all up once and you're holding ₹26. Now one person says "give me my ₹2 back, I'm leaving" and sends a substitute who hands you ₹1 instead. Do you recount all the money from scratch? Of course not — you simply do:

```
newBalance = oldBalance - 2 (returned) + 1 (new contribution) = 26 - 2 + 1 = 25
```

A bank does exactly this too: it never re-counts your whole balance for one transaction — it just adjusts it. Sliding Window uses the exact same idea: compute the first window's sum once, then only adjust for the one element leaving and the one element entering on every slide.

3.4 Applying Step 3 & 4 — Step-by-Step Walkthrough

Array: [100, 200, 300, 400] with indices 0, 1, 2, 3. $K = 2$.

Step 3 — find the FIRST window's information (sum) directly: low = 0, high = $K - 1 = 1$.

```
Window 1 = arr[0..1] = [100, 200]
sum = 100 + 200 = 300
```

Step 4 — slide low and high together, one step at a time, reusing the previous sum:

low, high	Window	Update Formula	Sum	Running Max
0, 1	[100, 200]	(first window — computed directly)	300	300
1, 2	[200, 300]	$300 - 100 + 300$	500	500
2, 3	[300, 400]	$500 - 200 + 400$	700	700

After the window [300, 400], high would become 4 — which is outside the array ($n = 4$). The loop stops there.

Result

Maximum Sum = 700 (from window [300, 400])

3.5 C++ Code

C++ — Fixed Window | Time: $O(N)$ | Space: $O(1)$

```
#include <bits/stdc++.h>
using namespace std;

int maxSumFixedWindow(vector<int>& arr, int k) {
    int n = arr.size();
    int low = 0, high = k - 1;

    // Step 3: information of the very first window – computed directly
    int sum = 0;
    for (int i = low; i <= high; i++) sum += arr[i];

    int result = sum;          // record the first window's info

    // Step 4: slide the window and update info each time
    while (high < n) {
        result = max(result, sum); // use the current window's info

        low++;
        high++;

        sum -= arr[low - 1]; // FIRE: remove the element that left

        if (high == n) break; // bounds check BEFORE adding (see note below!)

        sum += arr[high];    // HIRE: add the element that entered
    }

    return result;
}

int main() {
    vector<int> arr = {100, 200, 300, 400};
    cout << "Maximum sum = " << maxSumFixedWindow(arr, 2) << endl;
    return 0;
}
```

Output:

```
Maximum sum = 700
```

3.6 🐞 A Real Bug Caught Live — The $K = 1$ Edge Case

This is a genuinely useful debugging lesson, not just theory. Watch what happens if you check the array bound AFTER adding the new element instead of BEFORE:

What NOT to do

```
// ✗ BUGGY ORDER
sum -= arr[low - 1];
sum += arr[high];      // <-- runs even when high might be out of bounds!
if (high == n) break;  // <-- checked too late
```

With $K = 1$, low and high move together and high can reach n one step earlier than you'd expect — so `arr[high]` gets accessed when `high == n`, which is out of bounds (undefined behaviour / crash).

✓ The Fix

Always SUBTRACT first (always safe — the leaving index was always valid).
Then CHECK if `high == n` and break BEFORE you ever try to ADD `arr[high]`.
Order: subtract → check bounds → add. Never add before checking bounds.

This is exactly why dry-running edge cases (especially $K = 1$, or an empty array) by hand BEFORE trusting your code is such a valuable habit — bugs like this are easy to miss by just reading the code once.

4. Problem 2 — Minimum Size Subarray Sum (Variable Window)

🔗 Problem Statement

Given an array of POSITIVE integers and a target value, find the length of the SHORTEST contiguous subarray whose sum is $\geq$ target.

If no such subarray exists, return 0.

Array: [1, 2, 4, 4] Target = 4

4.1 Why This Is Harder — No Size Given

Search the problem statement as carefully as you like — there is no mention of a fixed length anywhere. It doesn't say "subarray of size 2" or "size 3". It just says "subarray", and the answer could turn out to be of ANY length. That single missing detail is what makes this a VARIABLE window problem instead of a fixed one — and noticeably harder for beginners.

⚠️ Critical Rule for Every Variable Window Problem

Always start with low = 0 AND high = 0 (never anything else).

Why? Because no matter how small the final answer turns out to be, a subarray must contain at least 1 element — so you always begin by looking at the smallest possible window and growing from there.

4.2 The Hire & Fire Analogy

Imagine a company that has no idea how many employees it takes to get a job done. It just starts HIRING people one at a time:

- Hire person 1 → check if the job is getting done. Not yet → hire another.
- Hire person 2 → still not enough → hire another.
- Hire person 3 → now the job IS getting done!

The moment the job starts getting done, the company gets greedy and starts FIRING people from the very first one hired, one at a time — to check if fewer people are actually enough:

- Fire the 1st person hired → job still getting done with fewer people? Great, note this down — it's a smaller team.
- Keep firing → as long as the job still gets done, keep noting the smaller size.
- The moment firing ONE more person breaks the job, stop firing — and go back to hiring the next new person.

That's the entire algorithm. Hiring = expanding the window from the right (high++). Firing = shrinking the window from the left (low++). The team size at any moment = the window's current length.

Action	Sliding Window Equivalent	Pointer Involved
Hire a new person	Expand window — add a new element on the right	high
Fire the oldest person	Shrink window — remove the oldest element on the left	low

4.3 Step-by-Step Walkthrough

Array: [1, 2, 4, 4] with indices 0, 1, 2, 3. Target = 4. low = 0, high = 0, sum = 0, result = ∞ (infinity, since we want a MINIMUM).

💡 Why Start result at Infinity?

Whenever you need a MINIMUM answer, start result at +infinity (INT_MAX in C++) — any real value found will be smaller, so it always gets accepted on the first comparison.

Whenever you need a MAXIMUM answer, start result at -infinity (INT_MIN in C++) instead — the mirror-image rule.

high	Action	Sum After	sum $\geq$ target?	Window / Length	Result
0	Hire arr[0]=1	1	No	—	∞
1	Hire arr[1]=2	3	No	—	∞
2	Hire arr[2]=4	7	Yes	[1,2,4] len=3	3
2	Fire arr[0]=1 (low→1)	6	Yes	[2,4] len=2	2
2	Fire arr[1]=2 (low→2)	4	Yes	[4] len=1	1
2	Fire arr[2]=4 (low→3)	0	No — stop firing	—	1
3	Hire arr[3]=4	4	Yes	[4] len=1	1
3	Fire arr[3]=4 (low→4)	0	No — stop firing	—	1

After this, high++ makes high = 4 = n, so the loop ends (no more elements to hire).

🏠 Result

Minimum length = 1 (the single element [4] alone already meets the target)

4.4 C++ Code

C++ — Variable Window | Time: $O(N)$ | Space: $O(1)$

```
#include <bits/stdc++.h>
using namespace std;

int minSubarrayLen(vector<int>& arr, int target) {
    int n = arr.size();
    int low = 0, high = 0;
    int sum = 0;
    int result = INT_MAX;    // we want a MINIMUM, so start at +infinity

    while (high < n) {
        sum += arr[high];    // HIRE: bring the new element into the window

        // While the team is doing the job, keep firing from the left
        while (sum >= target) {
            int length = high - low + 1;
            result = min(result, length);

            sum -= arr[low];    // FIRE: remove the oldest element
            low++;
        }

        high++;    // move on to hire the next person
    }

    return (result == INT_MAX) ? 0 : result;    // 0 if no valid subarray found
}

int main() {
    vector<int> arr = {1, 2, 4, 4};
    cout << "Minimum length = " << minSubarrayLen(arr, 4) << endl;
    return 0;
}
```

Output:

```
Minimum length = 1
```

Why Two while Loops and Not One?

The OUTER while ($high < n$) is the hiring loop — it always runs once per element, moving high forward.

The INNER while ($sum \geq target$) is the firing loop — it can run 0, 1, or many times per outer step, because once the job is "done", you keep shrinking greedily until it breaks again.

5. Fixed Window vs Variable Window — Coding Template

Aspect	Fixed Window (Problem 1)	Variable Window (Problem 2)
Starting point	low = 0, high = K - 1	low = 0, high = 0
First window's info	Computed by looping low → high once	Built up element by element as high grows
Expanding ("Hire")	Happens automatically — low & high move together	high++ every outer-loop step, sum += arr[high]
Shrinking ("Fire")	Happens automatically — low & high move together	Only while a condition holds (inner while loop), sum -= arr[low]
Loop structure	Single while loop	Outer while (expand) + inner while (shrink)
When to stop	high == n (array exhausted)	high == n (array exhausted)
Result update	Once per outer step	Once per inner (shrink) step — can be multiple times per outer step

6. Common Mistakes & Pro Tips

🚫 Mistakes to Avoid

- **Starting a variable window anywhere other than $low = high = 0$:** the smallest possible subarray always has length 1, so you must start there, never jump ahead.
- **Adding before checking array bounds:** always SUBTRACT first (it's always safe), THEN check if high has gone out of bounds, THEN add. Reversing this order causes out-of-bounds bugs — especially visible with $K = 1$.
- **Using an if instead of a while for the shrink condition:** in Problem 2, firing can happen multiple times in a row for the SAME hire — an if would only fire once and miss smaller valid windows.
- **Starting result at 0 for a minimum-length problem:** 0 will incorrectly "win" against every real answer. Always start at `INT_MAX` for minimums and `INT_MIN` for maximums.
- **Forgetting the "no valid subarray" case:** if result never gets updated from `INT_MAX`, the correct answer to return is 0, not `INT_MAX` itself.

💡 Pro Tips

- Think like a human first, code like a computer second — manually trace 2-3 slides on paper with real numbers before writing any loop.
- For Fixed Window: both pointers ALWAYS move together — if you ever find yourself moving only one of them, you've accidentally turned it into a variable window.
- For Variable Window: high always drives growth, low always drives shrink — never the other way around.
- Whenever the problem says "minimum"/"shortest", default to `INT_MAX` as the starting result. Whenever it says "maximum"/"longest", default to `INT_MIN`.

7. Complexity Cheat-Sheet

Problem	Window Type	Time Complexity	Space Complexity
Maximum Sum Subarray of Size K	Fixed	$O(N)$	$O(1)$
Minimum Size Subarray Sum	Variable	$O(N)$ — amortized (low moves at most N times total across the whole run)	$O(1)$

Even though Problem 2 has a nested while loop, the total work is still $O(N)$ — not $O(N^2)$ — because the inner loop's pointer (low) never resets; it only ever moves forward, and across the ENTIRE run it moves at most N times in total. This "amortized $O(N)$ " reasoning is a very common interview talking point.

8. Practice Problems

Fixed Window (same template as Problem 1)

4. First negative number in every window of size K
5. Maximum of all subarrays of size K (sliding window maximum)
6. Average of all contiguous subarrays of size K
7. Count occurrences of anagrams of a pattern in a string

Variable Window (same template as Problem 2)

8. Longest substring without repeating characters
9. Longest subarray with sum less than or equal to K
10. Smallest subarray with a given sum (general version — try negative numbers too, and notice why it gets trickier)
11. Minimum window substring
12. Longest substring with at most K distinct characters

9. Interview Q&A

Q: What are the 4 steps to solve any sliding window problem?

A: 1) Identify whether it's truly a sliding window problem. 2) Determine if the window is fixed-size or variable-size. 3) Compute the information (sum/max/min/etc.) of the very first window directly. 4) Slide the window — expanding and/or shrinking as needed — updating the information from the previous window's result each time, until the array is exhausted.

Q: How do you tell if a window should be fixed or variable?

A: If the problem explicitly states a size or length (like "subarray of size K"), it's a fixed window — both pointers move together. If no size is given and the problem instead asks for the longest/shortest/smallest valid window, it's a variable window — the pointers move independently based on a condition.

Q: Why does a variable window always start with low = 0 and high = 0?

A: Because the smallest possible subarray is always of length 1. Starting both pointers at index 0 means the window begins as small as it can possibly be, and is then grown one element at a time — guaranteeing no valid (even very short) answer is skipped.

Q: In the Minimum Size Subarray Sum problem, why is the shrink step inside a while loop instead of an if?

A: Because a single hire (adding one element) can sometimes make the condition true by a large margin, allowing MULTIPLE firings in a row before the condition breaks again. An if would only check and fire once, potentially missing a smaller valid window that's still hiding inside the current one.

Q: What value should you initialize your result variable to, and why does it matter?

A: For a minimum answer, initialize to INT_MAX (+infinity) so that the first real value found is always smaller and gets accepted. For a maximum answer, initialize to INT_MIN (-infinity) for the mirror reason. Starting at 0 is a common bug — it can incorrectly beat every genuine answer in a minimum-length problem.

Q: Why is the order of subtract-then-check-then-add important when sliding a fixed window?

A: Subtracting the element that left the window is always safe, since that index was always valid. But adding the new element entering on the right can go out of bounds if the window has reached the end of the array. Checking the bound after subtracting but before adding avoids accessing an invalid index — this matters especially for edge cases like $K = 1$.

Q: What is the time complexity of the variable window approach, given it has a nested while loop?

A: Still $O(N)$, not $O(N^2)$. The outer loop's pointer (high) moves forward at most N times, and the inner loop's pointer (low) also only ever moves forward and never resets — so across the ENTIRE execution, low moves at most N times too. This is called amortized analysis: the total combined work across both loops is bounded by $O(N)$.

Q: What should Minimum Size Subarray Sum return if no valid subarray exists?

A: 0. This happens when the result variable never gets updated away from its initial `INT_MAX` value — meaning the condition ($\text{sum} \geq \text{target}$) was never satisfied even using the entire array. The code should explicitly check for this and return 0 instead of the placeholder `INT_MAX`.

10. One-Page Quick Revision

Sliding Window — Day 2 Cheat Sheet

1. The 4 Steps: Identify → Fixed or Variable? → Info of first window → Slide & update info.
2. Fixed Window: low = 0, high = K-1 to start. Both pointers ALWAYS move together. Subtract → check bounds → add.
3. Variable Window: low = 0, high = 0 to start. high expands (Hire), low shrinks (Fire) — shrink in a while loop, not an if.
4. Minimum problems: start result at INT_MAX, return 0 if it never updates. Maximum problems: start result at INT_MIN.
5. Both patterns run in $O(N)$ time and $O(1)$ space — the variable window's nested loop is still $O(N)$ total, by amortized analysis.

Next up: more variable-window problems involving frequency maps (distinct characters, anagrams) — the same Hire & Fire intuition, just with a hashmap tracking what's inside the window instead of a single running sum.